

System Design Interview Question: Design Spotify

#93: System Design Interview (13 Minutes)



NEO KIM AND HAYK

OCT 06, 2025 PAID

Get my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post will help you prepare for the system design interview.

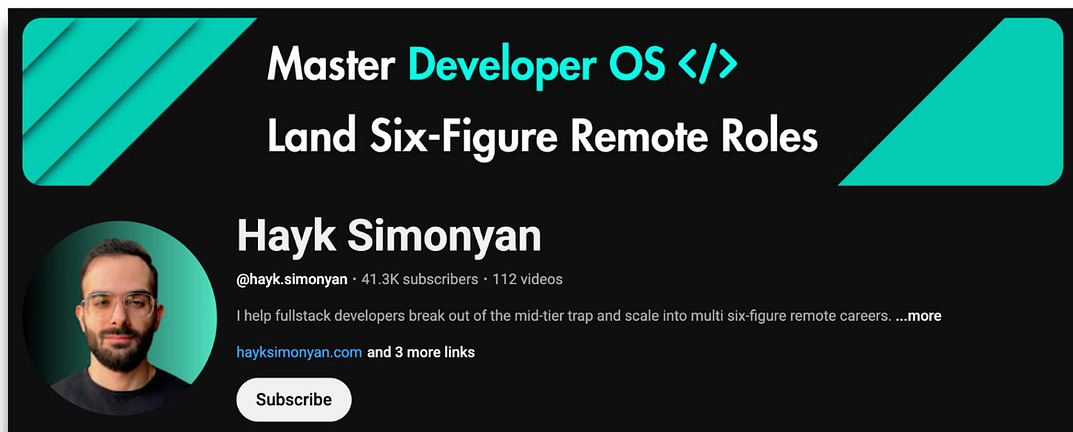
- *Share this post & I'll send you some rewards for the referrals.*

Building a music streaming platform like Spotify is a classic system design problem.

It includes audio delivery, metadata management, and everything in between.

Let's figure out how to design it during a system design interview.

I want to introduce Hayk Simonyan as a guest author.



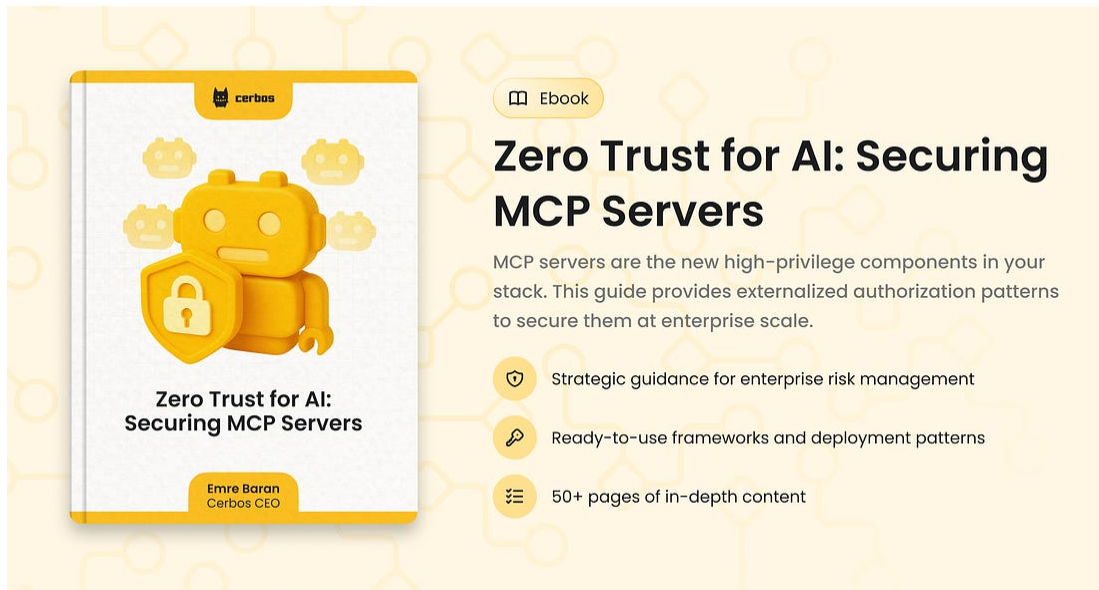
He's a senior software engineer specializing in helping mid-level engineers break through their career plateaus and secure senior roles.

If you want to master the essential system design skills and land senior developer roles, I highly recommend checking out Hayk's **YouTube channel**.

His approach focuses on what top employers actually care about: system design expertise, advanced project experience, and elite-level interview performance.

Onward.

Zero Trust for AI: Securing MCP Servers eBook by Cerbos (Sponsor).



MCP servers are becoming critical components in AI architectures, but they are creating a fundamental new risk that traditional security controls weren't designed to address.

Left unsecured, they're a centralized point of failure for data governance.

This **eBook** will show you how to secure MCP servers properly, using externalized, fine-grained authorization. Inside the ebook, you will find:

- How MCP servers fit into your broader risk management and compliance framework
- Why MCP servers break the traditional chain of identity in enterprise systems
- How role-based access control fails in dynamic AI environments
- Real incidents from Asana and Supabase that demonstrate these risks
- The externalized authorization architecture (PEP/PDP) that enables Zero Trust for AI systems

Get the practical blueprint to secure MCP servers before they become your biggest liability.

Download FREE eBook

Requirements & Assumptions

We're looking at roughly 500k users listening to about 30 million songs. The core requirements include:

- Artists can upload their songs.
- Users can search and play songs.
- Users can create and manage playlists.
- Users can maintain profiles.
- Basic monitoring and observability (health checks, error tracking, performance metrics).

Nothing fancy yet.

For audio formats, we use Ogg and AAC files¹ with different bitrates² for adaptive streaming³. For example,

- 64kbps for mobile data saving,
- 128kbps for standard quality,
- 320kbps for premium users.

On average, one song file at normal audio quality (standard bitrate) takes about 3MB of storage.

While the main constraints are fast playback start times, minimal rebuffering, and straightforward operations. We handle rebuffering using adaptive quality switching.

Capacity Planning

Let's crunch some numbers to understand what we're working with:

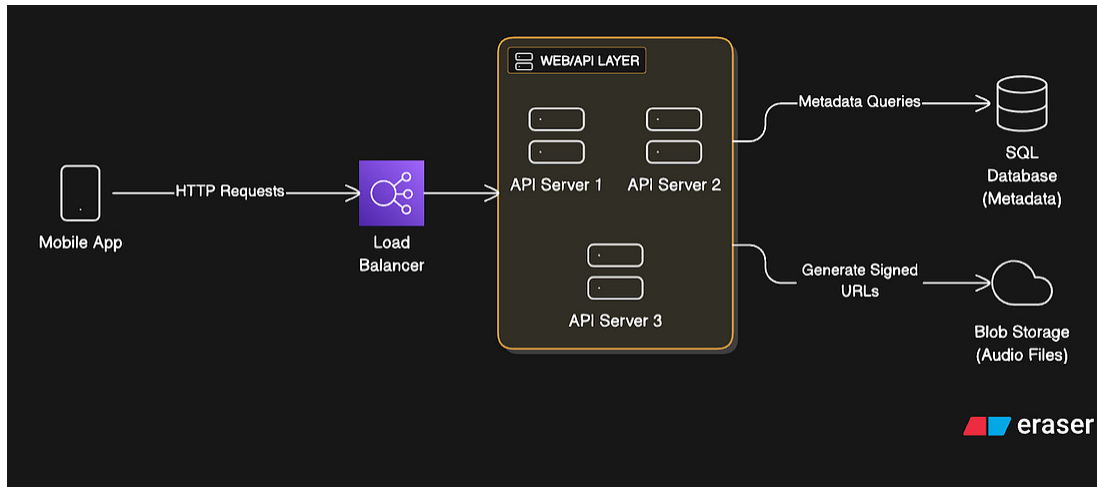
- **Song storage:**
 - $3\text{MB} \times 30\text{M songs} \approx 90\text{TB}$ of raw audio data.
 - This doesn't include replicas across different regions.
 - It also doesn't include versioning overhead when artists re-upload songs.
 - That's why we're looking at 2-3x this amount.
- **Song metadata:**
 - Each song needs a title, artist references, duration, file URLs, and so on.
 - At roughly 100 bytes per song $\times 30\text{M songs} \approx 3\text{GB}$.
 - That's not so much compared to the audio.
- **User metadata:**
 - User profiles, preferences, and playlist data are $\sim 1\text{KB} \times 500\text{k users} \approx 0.5\text{GB}$.
- **Daily bandwidth:**
 - The average listen time is 3.5 minutes at 128-160kbps; that's roughly 3-4MB per stream.
 - Let's assume each user streams 10-15 songs daily.
 - This leads to significant egress costs⁴.

The key insight is that audio dominates both storage costs and bandwidth. The metadata is just a small part in comparison.

Let's dive in!

Spotify System Design: High Level Architecture

The architecture breaks down into these key components:



1. Mobile App (Client)

The user-facing application handles UI, search, playback controls, and playlist management.

- It makes REST API⁵ calls to fetch metadata and manages local playback state.
- Client streams audio directly from blob storage⁶ or CDN⁷ using signed URLs⁸.
- And cache recently played songs locally for faster playback on the same song replay.
- When things go wrong, it uses retry logic⁹ for API calls.

The client handles network interruptions gracefully by pausing playback until connectivity returns.

2. Load Balancer

The load balancer spreads incoming requests across many API servers to prevent server overload.

- It could use round-robin or least-connections algorithms.¹⁰
- Also it performs health checks every 30 seconds and removes unhealthy instances from rotation.
- This is essential for managing traffic spikes during album releases.

Besides, it provides high availability against server failures and enables zero-downtime deployments¹¹.

3. Web/API Layer

The application servers are stateless; they handle business logic, authentication, and data access.

- They validate JWT tokens¹² and query the database for song metadata.
- They generate signed URLs for audio access and also save user actions for analytics.
- For reliability, they implement circuit breakers for database connections. Circuit breakers stop requests to failing services, preventing cascading failures.
- They use connection pooling to manage resources. This means reusing database connections instead of creating new ones for each request.

They also provide fallback responses for non-critical features when dependencies are down.

4. Blob Storage

Object storage systems like AWS S3¹³ could hold all the audio files.

- Files are organized in a hierarchical structure like
/artist/album/song.ogg

- They're accessed via signed URLs that expire after a few hours for security.
- Blob storage offers virtually unlimited scalability. It also comes with built-in durability and cost-effectiveness for large files. They replicate the storage across many regions for durability.

However, it has higher latency than local storage and potentially higher egress costs. We could instead use a distributed file system¹⁴ like Hadoop Distributed File System (**HDFS**). But blob storage is more managed and reliable for most use cases.

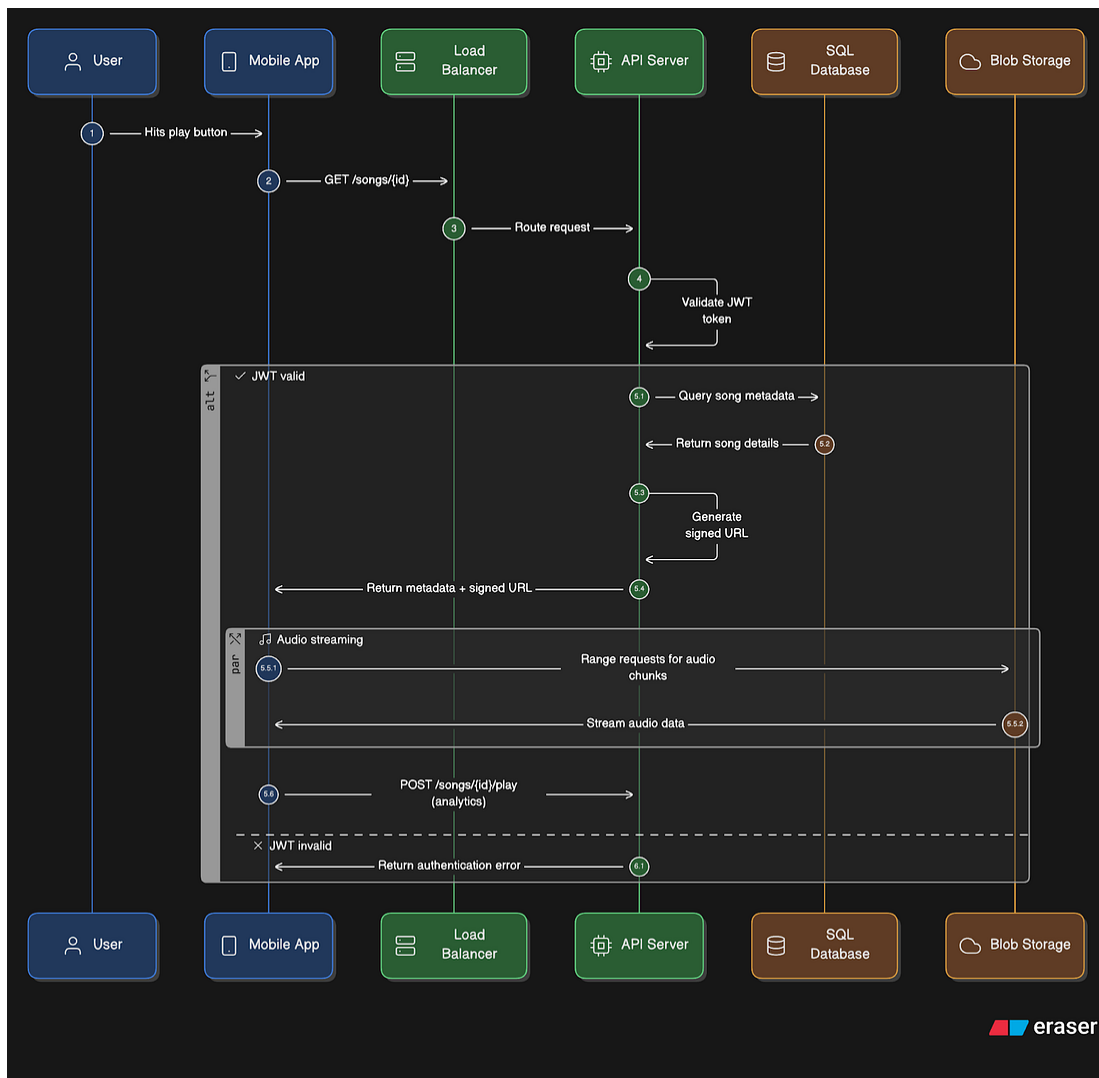
System Workflow

Now that we've covered the high-level architecture, let's explore the request workflow:

Read Workflow

Here's what happens when the user **plays a song**:

1. User hits play → App sends a GET request `/songs/{id}`
2. API authentication → API server validates the JWT token.
3. Metadata lookup → API server queries SQL database for song details (metadata).
4. URL generation → API server creates a signed URL for blob storage access.
5. Audio streaming → App fetches chunks of audio (range requests¹⁵) directly from blob storage using HTTP-based adaptive streaming (HLS or DASH¹⁶), which allows smooth playback, automatic bitrate switching, and broad device compatibility.
6. Analytics → App periodically calls POST `/songs/{id}/play` to track plays and listening time.



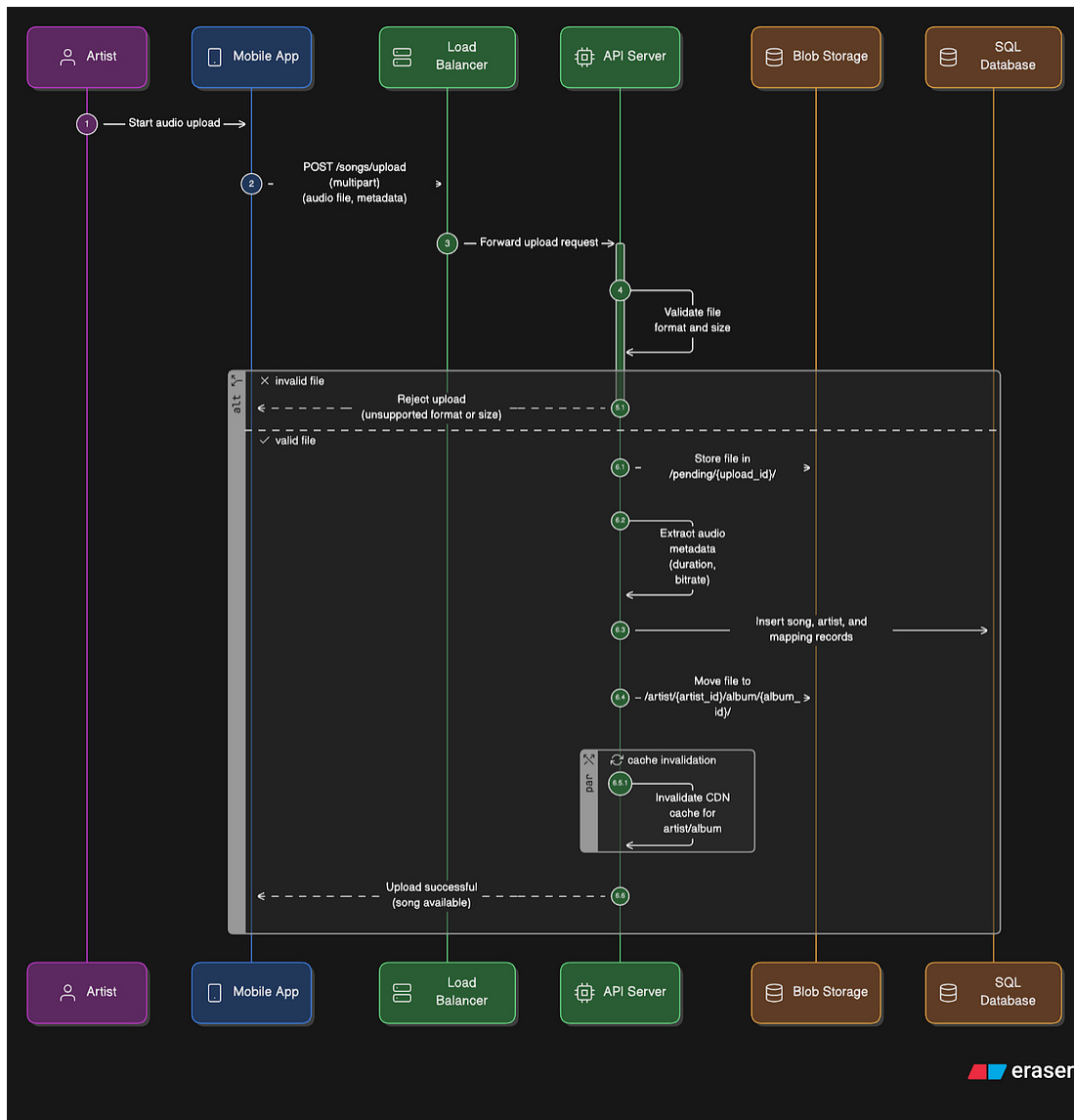
Write Workflow

Here's what happens when the **artist uploads a song**:

1. Artist uploads a song → The app sends it to the server using `/songs/upload` POST request with multipart form¹⁷ data.
2. File validation → API server checks its format, duration, and file size limits.
3. Blob storage → API server uploads file to `/pending/{upload_id}/song.ogg`
4. Metadata processing → API server triggers a separate background service to read the uploaded file and extract details like duration,

bitrate, and also generate preview images.

5. Database insert → API server then creates entries in these tables:
 - Songs: adds the new song record.
 - Artists: adds the artist details if not already there.
 - ArtistSongs: links the artist to the song.
6. File promotion → The system moves the song from pending to its final, organized location `/artist/{id}/album/{id}/song.ogg`
7. CDN invalidation → CDN stores cached copies of content (like artist pages, album metadata, song details) closer to users for fast delivery. It clears old cached data so users see the latest songs and artist updates right away.



API Design

Our core endpoints would look something like:

Search & Discovery

- Search different content types with pagination
 - GET /search?q={query}&type=song,artist&limit=20&offset=0
- Get trending songs, optionally filtered by genre

- GET /songs/trending?genre={genre}&limit=50
- Get all songs by a specific artist
 - GET /artists/{id}/songs?limit=50

Content Access

- Get song metadata and streaming URL
 - GET /songs/{id}
- Direct streaming endpoint (alternative to signed URLs)
 - GET /songs/{id}/stream
- Get playlist details with an optional song list
 - GET /playlists/{id}?include_songs=true

User Actions

- Create a new playlist
 - POST /playlists

```
{
  "name": "My Favorites",
  "is_public": false
}
```

- Add songs to the playlist
 - PUT /playlists/{id}/songs

```
{
  "song_ids": [123, 456, 789],
  "position": 5
}
```

- Remove a song from the playlist
 - DELETE /playlists/{id}/songs/{song_id}
- Like/unlike a song
 - POST /songs/{id}/like

User Management

- Get the current user's playlists
 - GET /users/me/playlists
- Get songs liked by the user
 - GET /users/me/liked-songs?limit=50&offset=0
- Follow an artist
 - POST /users/me/follow/{artist_id}

Most endpoints return JSON with a consistent structure. The JSON includes metadata such as total_count, limit, offset for pagination, and proper HTTP status codes¹⁸.

And the app sends the JWT token in the Authorization header for authentication.

Data Storage

This is where we split responsibilities between two storage types:

Blob Storage

The system uses blob storage to **store audio files**.

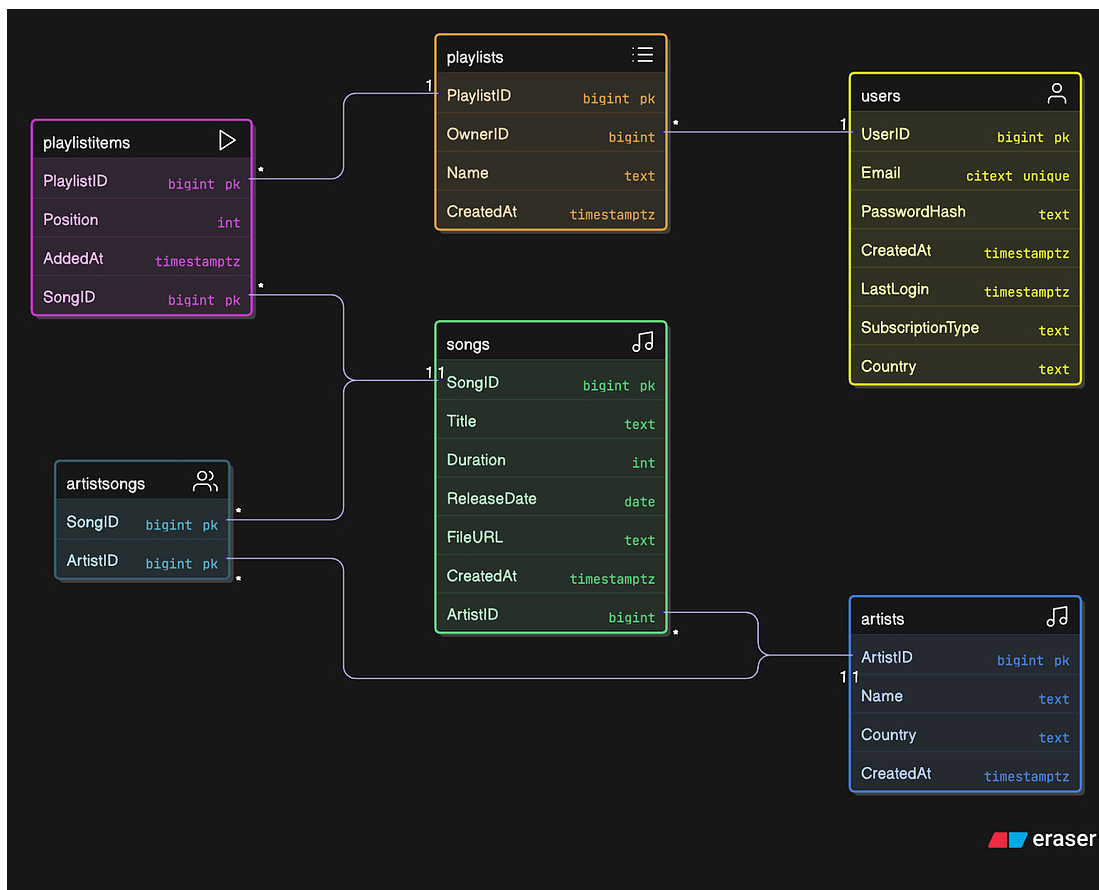
- The audio files are immutable; they rarely change once uploaded.
- We'd organize them with a sensible folder structure
 - /artist/{artistId}/album/{albumId}/{songId}.ogg

This makes it easy to manage uploads and serves as your source of truth for audio content.

Relational Database

The system uses an SQL database, such as PostgreSQL, to **store metadata** with strong consistency.

The core tables include:



- Users: stores information about each user.

```
CREATE TABLE Users (  
  UserID      BIGINT PRIMARY KEY,  
  Email       CITEXT UNIQUE NOT NULL,  
  PasswordHash TEXT NOT NULL,  
  CreatedAt   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  LastLogin   TIMESTAMPTZ  
);
```

- Artists: stores information about each artist.

```
CREATE TABLE Artists (  
  ArtistID    BIGINT PRIMARY KEY,  
  Name        TEXT NOT NULL,  
  Country     TEXT,  
  CreatedAt   TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

- Songs: stores information about each song.

```
CREATE TABLE Songs (  
  SongID      BIGINT PRIMARY KEY,  
  Title       TEXT NOT NULL,  
  ArtistID    BIGINT NOT NULL REFERENCES Artists(ArtistID),  
  Duration    INT NOT NULL,  
  ReleaseDate DATE,  
  FileURL     TEXT NOT NULL,  
  CreatedAt   TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

- Playlists: stores information about playlists created by users.

```
CREATE TABLE Playlists (
  PlaylistID BIGINT PRIMARY KEY,
  OwnerID    BIGINT NOT NULL REFERENCES Users(UserID),
  Name       TEXT NOT NULL,
  CreatedAt  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

- PlaylistItems: stores the songs inside a playlist.

```
CREATE TABLE PlaylistItems (
  PlaylistID BIGINT REFERENCES Playlists(PlaylistID),
  SongID     BIGINT REFERENCES Songs(SongID),
  Position   INT NOT NULL,
  AddedAt    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (PlaylistID, Position)
);
```

- ArtistSongs: a junction table that stores relationships between songs and artists. For example, there could be many-to-many relationships with a song by many artists or many songs by a single artist.

```
CREATE TABLE ArtistSongs (
  ArtistID BIGINT REFERENCES Artists(ArtistID),
  SongID   BIGINT REFERENCES Songs(SongID),
  PRIMARY KEY (ArtistID, SongID)
);
```

We chose SQL because we need JOINS¹⁹ for complex queries.

- For example, to find all songs by artists that a user follows.

Also, you want referential integrity to prevent orphaned records. It means ensuring every record in one table points to a valid record in another table, so there are no orphan entries with missing links.

We can optimize common queries by adding database indexes²⁰ on frequently accessed columns.

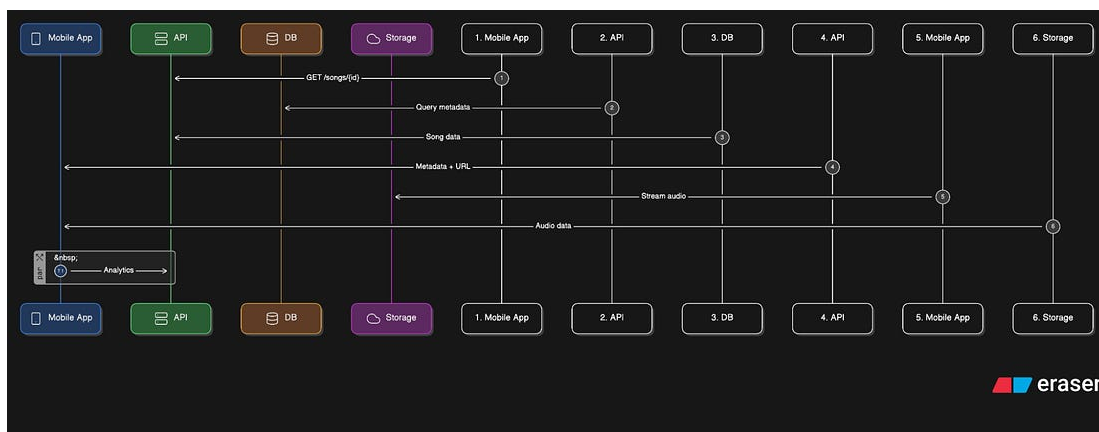
```
CREATE INDEX idx_songs_title_trgm ON Songs USING GIN (Title gin_trgm_ops);  
CREATE INDEX idx_artists_name_trgm ON Artists USING GIN (Name gin_trgm_ops);  
CREATE INDEX idx_playlistitems_song ON PlaylistItems (SongID);
```

The trigram indexes²¹ on song titles and artist names enable fast search capabilities, handling typos and partial matches that users expect.

And the playlist items index speeds up queries when fetching songs for a specific playlist or finding which playlists contain a specific song.

Putting It All Together

Here's the full user journey when they **play a song**:



1. App sends a GET /songs/{id} request to the API.

2. API queries the SQL database for song metadata.
3. The API returns metadata along with a signed URL pointing directly to blob storage or proxies the audio stream itself.
4. The client makes range requests to stream the audio in chunks.
5. App periodically sends progress information to the API for analytics.

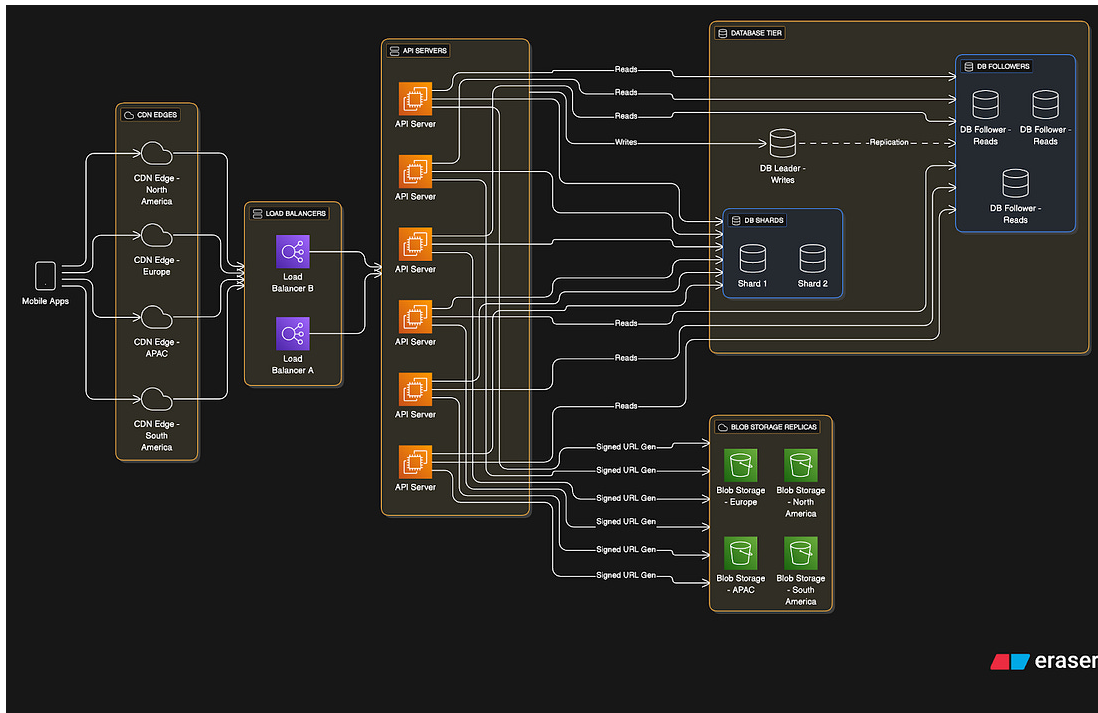
The signed URL approach is better because it takes the load off the API servers. But the proxy approach gives us more control over access patterns.

Scalability

When we hit real scale, the numbers get much bigger than in the initial phase:

- We have **200 million songs and 50 million users**
- Song metadata: $100 \text{ bytes} \times 200\text{M songs} \approx 20 \text{ GB}$
- User metadata: $1\text{KB} \times 50\text{M users} \approx 50 \text{ GB}$
- Audio storage: $3\text{MB} \times 200\text{M songs} \approx 600 \text{ TB}$ (before replicas and regional copies)

Now, we need to make some major architectural changes:



CDN for Audio Delivery

Popular songs get cached at edge locations worldwide to reduce latency for users. For example, a song cached in Germany serves much faster to European users than fetching it from the origin storage in the United States.

We could implement least recently used (**LRU**) eviction for cache management. LRU automatically removes the least accessed content when storage fills up. And the cache would fetch from the origin storage on a cache miss.

The CDN fetches content from blob storage using signed URLs, which allows it to serve the content securely.

The major advantage is a better user experience with fast load times and reduced capacity costs. You're not serving every stream from your origin servers.

Database Scaling

We need to introduce **leader-follower replication**²² to scale the database. Followers handle read traffic like searches and metadata lookups. While the leader handles writes like new uploads and playlist changes.

This approach works well because music streaming is heavily read-biased. Users search and browse far more than they upload or change playlists.

The major advantage is that you can scale read capacity by adding more followers. It also provides automatic failover if the leader goes down.

However, we might deal with replication lag²³. Followers could be slightly behind the leader. Also, there's increased complexity from managing many database instances. We also face potential consistency issues where a user might not immediately see their own writes if routed to a follower.

When catalog growth or hot partitions²⁴ become an issue, we'll need to consider sharding strategies. **Sharding** splits data across many database instances. For example, we could shard users by geography. US users go on one shard, EU users on another. Or we partition songs by artist ID ranges.

This gives us horizontal scalability and improves performance by keeping related data together. But the downside is that cross-shard queries become expensive or impossible. We lose some SQL features like foreign keys across shards. Besides, rebalancing data on growth becomes a complex operational challenge.

For a music platform, we'd start with geographic sharding. Most users primarily listen to artists from their region. However, you'd need a strategy for global artists whose songs are popular everywhere.

API Layer Horizontal Scaling

Morning commutes and evening listening sessions create predictable traffic spikes. So it's necessary to set up many stateless API servers behind the load balancer to handle varying loads throughout the day.

These servers don't maintain any user session state and use JWT tokens.

Adding capacity is straightforward.

We just spin up more instances, and the load balancer automatically includes them. This approach is cost-effective. Scale up during peak hours, then scale down overnight. It provides fault tolerance. If one server crashes, the others continue serving. It also enables rolling deployments²⁵ without downtime.

The main tradeoff is that keeping servers truly stateless requires careful design.

You can't store anything locally between requests. This means more database calls and potential performance issues.

Reliability & Operations

Let's also introduce monitoring to track system health and find performance issues.

- Health checks across all services with automatic traffic routing away from unhealthy instances.
- Retries with exponential backoff²⁶ for transient failures, especially for audio streaming, where network hiccups are common.

Here are some key metrics to watch:

- Time to first byte for audio streams

- Error rates across all endpoints
- CDN cache hit ratios (should be 85% + for popular content)
- Database query latency

Besides, implement a simple fallback strategy:

- If the CDN misses, then fetch from origin storage and populate the cache for next time.
- If database replicas are down, fall back to the leader with circuit breaker²⁷ protection.

The Bottom Line

Start with a simple design and scale incrementally. We don't over-engineer for problems we don't have yet, but the architecture naturally accommodates growth when it's needed.

👋 I'd like to thank [Hayk](#) for writing this newsletter!

- Also don't forget to join his [YouTube channel](#).

It'll help you master the essential system design skills, not just in theory but in practice, and land senior developer roles.

Subscribe to get simplified case studies delivered straight to your inbox:

✓ **Subscribed**



👋 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

Want to advertise in this newsletter? 📄

If your company wants to reach a 175K+ tech audience, [advertise with me](#).

Thank you for supporting this newsletter.

You are now 175,001+ readers strong, very close to 176k. Let's try to get 176k readers by 10 October. Consider sharing this post with your friends and get rewards.

Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share

-
- Block diagrams created with [Eraser](#)
-

- 1 OGG and AAC are audio file formats that compress music without losing much quality and take up less space.
- 2 A bitrate is the amount of data used to store or stream audio every second.
- 3 Adaptive streaming means the app automatically changes the quality of the audio stream depending on your internet speed.
- 4 Egress costs are the fees cloud providers charge when data leaves their network to reach users.
- 5 A REST API is a web service that lets clients and servers exchange data using standard HTTP methods (like GET, POST, PUT, DELETE).

- 6 Blob storage is a cloud service for storing large, unstructured files, such as audio, video, or images.
- 7 A content delivery network (**CDN**) is a global set of servers that deliver content to users from the closest server to reduce latency and load times.
- 8 Signed URLs are time-limited URLs that grant temporary access without exposing storage credentials.
- 9 It means that if an API request fails, the system automatically retries it until it succeeds or gives up after reaching a limit.
- 10 **Round-robin** sends requests to servers one after another in order. **Least-connections** sends requests to the server that has the fewest active connections.
- 11 Deployment means releasing new or updated software to servers so it becomes available for use.
- 12 A JSON Web Token (**JWT**) is an encrypted string used to securely pass user identity and permissions between a client and a server.
- 13 Amazon Simple Storage Service (**AWS S3**) is a storage service that stores and retrieves any amount of data as objects in buckets.
- 14 A distributed file system stores data across many servers and makes it appear as one big storage system, so large files can be split, stored, and processed in parallel.
- 15 A range request is an HTTP request where the client asks the server for only a specific part (byte range) of a file instead of downloading the entire file at once.
- 16 HTTP Live Streaming (**HLS**) and Dynamic Adaptive Streaming over HTTP (**DASH**) are adaptive streaming protocols that split audio and video into small chunks delivered over HTTP. They let the client switch between different quality levels based on network speed for smooth playback.
- 17 Multipart form data is an HTTP format that allows sending files and text fields

together in a single request, used for file uploads.

- 18 An HTTP status code is a three-digit number that tells the client whether an HTTP request was successful, failed, or needs extra action.
- 19 A SQL Join is a way to combine rows from two or more tables based on a related column between them.
- 20 A database index is a special data structure that makes searching and fetching rows quicker, like a lookup shortcut for specific columns.
- 21 A **trigram index** is a special database index that speeds up text search by breaking strings into groups of three characters (trigrams) to support fast matching and fuzzy search.
- 22 Leader–follower replication is a database setup where the leader handles writes, and followers copy the data to handle read requests.
- 23 Replication lag is the delay between when data gets written to the leader database and when it appears on the follower databases.
- 24 A hot partition is the part of a database that gets a high amount of traffic, causing performance bottlenecks.
- 25 Rolling deployments are a release strategy where new software versions are gradually rolled out to servers one by one, so the system stays online without downtime.
- 26 Retry with exponential backoff is a technique where a failed request gets retried with increasing wait times (for example, 1s, 2s, 4s, 8s) to reduce server overload.
- 27 The circuit breaker pattern is a design that stops requests to a failing service after repeated errors and only lets them try again after a cooldown period.